

Rapport Technique

Service Reviews - FlexShop

FlexShop Team

12 juin 2026

1 Présentation du projet

Ce document présente le service `reviews-service`, un microservice Symfony dédié à la gestion des avis clients pour la marketplace FlexShop.

1.1 Lien vers le dépôt Git

Le code source complet est disponible sur GitHub :

<https://github.com/GnsYnns/reviews-service>

2 Architecture et endpoints

2.1 Endpoints API

Méthode	Endpoint	Description
GET	/api/reviews	Liste paginée des avis
GET	/api/reviews/{id}	Détail d'un avis
POST	/api/reviews	Créer un avis
PUT	/api/reviews/{id}	Modifier un avis
DELETE	/api/reviews/{id}	Supprimer un avis
GET	/api/stats/reviews/average/{productId}	Moyenne des notes
GET	/api/stats/reviews/search?q=...	Recherche full-text

TABLE 1 – Endpoints du service Reviews

2.2 Swagger UI

La documentation interactive est accessible via Swagger UI :

<http://localhost:8000/api>

3 Difficultés rencontrées et solutions

3.1 Difficulté 1 : Conflit de port avec le service Catalogue

Problème : Le service Symfony (port 8000) interceptait les appels vers le Catalogue Django configuré sur le même port.

Solution : Modification de l'URL du Catalogue dans `ProductExistsValidator.php` :

```
private string $catalogUrl = 'http://localhost:8001/api/v1'
```

Le Catalogue Django doit maintenant tourner sur le port 8001.

3.2 Difficulté 2 : Conflit de routes API Platform

Problème : Les endpoints custom `/api/reviews/search` et `/api/reviews/average` entraient en conflit avec le routage d'API Platform, retournant des erreurs 404.

Solution : Changement du préfixe de route pour éviter le conflit :

```
#[Route('/api/stats')]
class ReviewStatsController extends AbstractController
```

Les endpoints sont maintenant accessibles via `/api/stats/reviews/...`

3.3 Difficulté 3 : Content-Type pour API Platform

Problème : API Platform rejette les requêtes avec `Content-Type: application/json`, exigeant `application/ld+json`.

Solution : Mise à jour des tests PHPUnit et documentation pour utiliser le bon header :

```
Content-Type: application/ld+json
```

3.4 Difficulté 4 : Configuration base de données test

Problème : PHPUnit tentait de se connecter à une base `reviews_test` qui n'existait pas (suffixe automatique Doctrine).

Solution : Suppression de la configuration `dbname_suffix` dans `config/packages/doctrine.yaml` :

```
when@test:
    doctrine:
        dbal:
            # dbname_suffix: suppress
```

3.5 Difficulté 5 : Cache HTTP pour le Catalogue

Problème : Chaque validation de produit déclenchait un appel HTTP vers le Catalogue, risquant de surcharger le service.

Solution : Implémentation d'un cache de 60 secondes :

```
$exists = $this->cache->get($cacheKey, function () use ($value) {
    $response = $this->httpClient->request(
        'GET',
        $this->catalogUrl . '/products/' . $value . '/',
        ['timeout' => 3]
    );
    return $response->getStatusCode() !== 404;
});
```

Pool de cache dédié dans `cache.yaml` :

```
catalog.cache:
    adapter: cache.app
    default_lifetime: 60
```

4 Collection Postman

Une collection Postman exportée est disponible dans le dossier `docs/` :

Fichier : `docs/FlexShop-Reviews-API.postman_collection.json`

Cette collection contient les requêtes suivantes :

- GET `/api/reviews` (liste paginée)
- GET `/api/reviews?productId=1` (filtrage)
- POST `/api/reviews` (création avec validation)
- GET `/api/stats/reviews/average/1` (moyenne)
- GET `/api/stats/reviews/search?q=excellent` (recherche)

5 Validation et tests

5.1 Tests PHPUnit

Les tests sont exécutés avec :

```
php bin/phpunit
```

Résultats des 3 tests implémentés :

- `testGetReviewsReturnsOk` - Vérifie le endpoint liste
- `testGetNonExistentReviewReturns404` - Vérifie la gestion des 404
- `testCreateReviewWithInvalidRatingFails` - Vérifie la validation (rating 1-5)

5.2 Points d'attention implémentés

- **Timeout HTTP** : 3 secondes sur les appels au Catalogue
- **Gestion Catalogue DOWN** : Try/catch avec message "Catalogue indisponible"
- **Validation custom** : `ProductExists` vérifie l'existence avant création
- **Cache Catalogue** : 60 secondes pour limiter les appels réseau

6 Conclusion

Le service Reviews est fonctionnel avec :

- API CRUD complète via API Platform
- Endpoints custom pour statistiques et recherche
- Validation métier avec vérification Catalogue
- Cache HTTP pour optimiser les performances
- Tests PHPUnit validant les cas principaux